OTEL-05: Metrics Instrumentation

> **Series:** OTEL — OpenTelemetry Integration | **Notebook:** 5 of 8 | **Created:** January 2026 | **Last Updated:** 10/02/2026

Creating Custom Metrics with OpenTelemetry

OpenTelemetry metrics provide quantitative measurements of your application's behavior over time. This notebook covers metric types, instrumentation patterns, and integration with Dynatrace.

Table of Contents

1. Metric Types
2. Instrument Selection
3. Creating Metrics
4. Metric Attributes
5. Aggregation and Views
6. Prometheus Integration
7. Best Practices

Prerequisites

Requirement	Details
Knowledge	OTEL-01 and OTEL-03
Environment	Collector with metrics pipeline

1. Metric Types

OTel Metric Instruments

Instrument	Aggregation	Use Case	Example
Counter	Sum (monotonic)	Cumulative events	Total requests
UpDownCounter	Sum (non-monotonic)	Current values	Active connections
Histogram	Distribution	Value ranges	Response time
Gauge	Last value	Current measurement	Temperature

Sync vs Async

Type	When Recorded	Use Case
Synchronous	On each operation	Request handling
Asynchronous (Observable)	On collection	System stats, gauges

2. Instrument Selection

Decision Guide

Question	If Yes	If No
Value only goes up?	Counter	UpDownCounter or Histogram
Measuring duration/size?	Histogram	Counter or UpDownCounter
Current state?	Gauge	Counter or Histogram
Recording in request path?	Sync	Async (Observable)

Common Patterns

Metric	Instrument	Why
Request count	Counter	Monotonically increasing
Request duration	Histogram	Distribution needed
Active requests	UpDownCounter	Goes up and down
Queue size	ObservableGauge	Current state
CPU usage	ObservableGauge	Periodic measurement

3. Creating Metrics

Python Examples

```
from opentelemetry import metrics
from opentelemetry.sdk.metrics import MeterProvider
from opentelemetry.sdk.metrics.export import PeriodicExportingMetricReader
from opentelemetry.exporter.otlp.proto.grpc.metric_exporter import
OTLPMetricExporter

# Setup
exporter = OTLPMetricExporter(endpoint="http://collector:4317")
reader = PeriodicExportingMetricReader(exporter, export_interval_millis=60000)
provider = MeterProvider(metric_readers=[reader])
metrics.set_meter_provider(provider)

# Get meter
meter = metrics.get_meter(__name__)

# Counter
request_counter = meter.create_counter(
    name="http.server.request.count",
    description="Total HTTP requests",
    unit="1"
)

# Histogram
request_duration = meter.create_histogram(
    name="http.server.request.duration",
    description="HTTP request duration",
    unit="s"   # the semantic convention's unit — seconds, not ms
)

# UpDownCounter
active_requests = meter.create_up_down_counter(
    name="http.server.active_requests",
    description="Active HTTP requests",
    unit="1"
)
```

> **Keep the semantic-convention unit.** `http.server.request.duration` is defined as a **Histogram in seconds (`s`)**. Auto-instrumentation libraries emit it in seconds; a hand-written instrument with the same name in milliseconds puts two units under one

> metric key, and every average or percentile across services becomes meaningless. Use your own metric name if you need different units.
>
> **Sources:** HTTP metrics semantic conventions (opentelemetry.io).

Using Metrics

```
import time

def handle_request(request):
    # Increment active requests
    active_requests.add(1, {"method": request.method})
    start = time.time()

    try:
        response = process_request(request)
        status = response.status_code
    except Exception:
        status = 500
        raise
    finally:
        # Record duration
        duration_s = time.time() - start
        request_duration.record(duration_s, {
            "method": request.method,
            "status": str(status)
        })

        # Increment counter
        request_counter.add(1, {
            "method": request.method,
            "status": str(status)
        })

        # Decrement active requests
        active_requests.add(-1, {"method": request.method})

    return response
```

Observable (Async) Metrics

```
import psutil

# Observable gauge for CPU
def get_cpu_usage(options):
    yield metrics.Observation(psutil.cpu_percent(), {})

cpu_gauge = meter.create_observable_gauge(
    name="system.cpu.usage",
    callbacks=[get_cpu_usage],
    description="CPU usage percentage",
    unit="%"
)

# Observable counter for network bytes
last_bytes = {"sent": 0, "recv": 0}

def get_network_io(options):
    net = psutil.net_io_counters()
    yield metrics.Observation(net.bytes_sent, {"direction": "sent"})
    yield metrics.Observation(net.bytes_recv, {"direction": "recv"})

network_counter = meter.create_observable_counter(
    name="system.network.io",
    callbacks=[get_network_io],
    description="Network I/O bytes",
    unit="By"
)
```

4. Metric Attributes

Common Attributes

Attribute	Example	Use
`http.request.method`	`GET`, `POST`	HTTP method breakdown (was `http.method`)
`http.response.status_code`	`200`, `500`	Response status (was `http.status_code`)

Attribute	Example	Use
`service.name`	`checkout-api`	Service identification
`environment`	`production`	Environment

Cardinality Considerations

High Cardinality (Avoid)	Low Cardinality (Good)
User ID	Region
Request ID	HTTP method
Timestamp	Status code class
Full URL	Route template

Attribute Best Practices

```
# Good - Low cardinality
request_counter.add(1, {
    "method": "GET",
    "route": "/api/users/{id}",  # Template, not actual ID
    "status": "2xx"  # Status class
})

# Bad - High cardinality
request_counter.add(1, {
    "user_id": "user-12345",  # Millions of values
    "url": "/api/users/12345",  # Every request different
    "timestamp": "2026-01-26T10:00:00"  # Every second different
})
```

Advanced OTLP metric dimensions changes which attributes you pay for

Everything above assumes Dynatrace keeps only some of your attributes as dimensions. Without the setting below, the *Allow list: resource and scope attributes* setting decides which resource and scope attributes become dimensions. One setting changes that. With **Advanced OTLP metric dimensions** turned on:

> *"Dynatrace ingests all resource, scope, and data-point attributes as metric dimensions by default, except for attributes on the Deny list: all attributes."*

The same page gives the consequence for existing content: *"dimension keys that were previously normalized are now ingested as-is. For example, ingested dimension keys will preserve uppercase characters. This can increase the cardinality of some metrics and may affect dashboards, charts, alerts, and saved queries that depend on specific dimension sets."* It also notes that *"Relaxed ingestion limits apply when this feature is enabled."*

Two practical consequences:

- **The cardinality table above now depends on your code, not on the allow list.** A high-cardinality data-point attribute that used to be dropped at ingest becomes a dimension. Use **Deny list: all attributes** to drop anything you do not want. Attribute names there are case-sensitive and must match the original name.
- **Turn it on deliberately, and early, if you are heading to the latest Dynatrace.** The ready-made *Check your upgrade readiness* dashboard lists this setting as a pre-upgrade opt-in, so that nothing built on today's dimension set changes unnoticed at upgrade time. Enable it, then check the dashboards and alerts that group or filter by OTLP dimensions, and look especially for keys whose case changed.

> **Sources:** Configure OTLP metrics ingestion (DT docs) — all quotes above. The pre-upgrade opt-in comes from the ready-made *Check your upgrade readiness* dashboard (OpenTelemetry metrics section, read 09/29/2026), which has no public URL.

5. Aggregation and Views

Histogram Buckets

Configure explicit bucket boundaries:

```
from opentelemetry.sdk.metrics import MeterProvider
from opentelemetry.sdk.metrics.view import View,
ExplicitBucketHistogramAggregation

# Custom histogram buckets for response times
duration_view = View(
    instrument_name="http.server.request.duration",
    aggregation=ExplicitBucketHistogramAggregation(
        boundaries=[0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10]
# seconds
    )
)

provider = MeterProvider(
    metric_readers=[reader],
    views=[duration_view]
)
```

Choosing Bucket Boundaries

The bucket boundaries decide what shape of latency distribution your histogram can describe. Pick them once — changing them later forces a metric reset and breaks dashboards built on the old layout.

Rule of thumb:

- 8–12 buckets, log-spaced.
- Span from `P50 / 10` to `P99 × 2` of observed latency.
- Add `+Inf` (implicit in OpenTelemetry; explicit in Prometheus client libraries).

Two realistic patterns:

Workload	Buckets (seconds)	Why this shape
HTTP request latency (sub-second SLO)	`0.1, 0.2, 0.5, 1.0, 2.0, 5.0, 10.0, +Inf`	Tight resolution under 1s where the SLO lives; tail tops out at 10s
Long-running event processor	`0.2, 0.4, 1.0, 2.0, 4.0, 16.0, 96.0, +Inf`	Spans two orders of magnitude; coarse at the tail because operators only care whether jobs exceed 96s

Cardinality cost. A histogram produces `(buckets + 2) × series` timeseries — one `_bucket{le=...}` per boundary (including `+Inf`), plus `_count` and `_sum`. A histogram

with 8 buckets broken down by `method` (5 values) and `status_code` (10 values) yields `10 × 50 = 500` timeseries **per pod**. Multiply by replicas and pod count; tens of thousands of series is not unusual.

Trim labels before adding buckets. The cardinality multiplier on labels is larger than on bucket count.

Drop Metrics via Views

```
from opentelemetry.sdk.metrics.view import DropAggregation

# Drop noisy metrics
drop_view = View(
    instrument_name="internal.debug.*",
    aggregation=DropAggregation()
)
```

6. Prometheus Integration

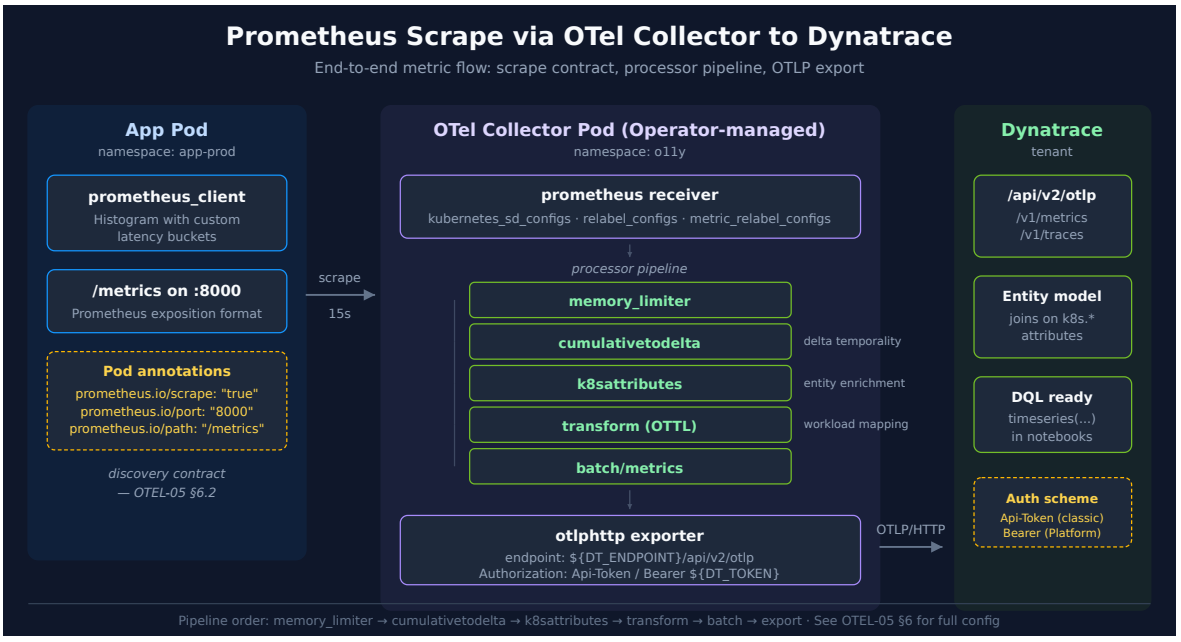
OpenTelemetry and Prometheus both expose metrics, and an OTel Collector can bridge them in either direction:

Direction	Pattern	When to use
Prometheus → OTLP	Collector `prometheus` receiver scrapes app; `otlphttp` exporter sends to Dynatrace	App already exposes `/metrics`; you want it in Dynatrace without changing app code
OTel SDK → Prometheus	App uses `PrometheusMetricReader`; existing Prometheus stack scrapes it	App is OTel-instrumented but an existing Prometheus-based monitoring stack must keep working

The first direction is the common path in Dynatrace deployments. The rest of this section walks the end-to-end pattern.

> **This section shows the single-collector pattern** — one collector scrapes a fixed set of annotated pods. It does not scale horizontally (a second replica would scrape every target again, double-counting). For thousands of targets or millions of data

> points per minute, use the tiered **Target Allocator** architecture in **OTEL-03 — Collector Deployment Patterns, §5 Scaling Prometheus Scraping** (scraper Deployment + gateway StatefulSet + resource-hash routing).



6.1. Step 1: Expose Prometheus Metrics from the App

The application uses `prometheus_client` and declares histograms with explicit buckets matching the workload's latency shape (see §5 *Choosing Bucket Boundaries*):

```
from prometheus_client import Histogram, start_http_server

# HTTP latency — sub-second SLO
request_latency = Histogram(
    "myapp_http_request_duration_seconds",
    "HTTP request duration in seconds",
    buckets=(0.1, 0.2, 0.5, 1.0, 2.0, 5.0, 10.0, float("inf"))
)

# Long-running processor — different distribution, different buckets
process_latency = Histogram(
    "myapp_event_processor_duration_seconds",
    "Event processor duration in seconds",
    buckets=(0.2, 0.4, 1.0, 2.0, 4.0, 16.0, 96.0, float("inf"))
)

# Expose /metrics on container port 8000
start_http_server(8000)
```

The exposition format produces one timeseries per bucket boundary plus `_count` and `_sum`:

```
myapp_http_request_duration_seconds_bucket{le="0.1"} 12
myapp_http_request_duration_seconds_bucket{le="0.2"} 47
myapp_http_request_duration_seconds_bucket{le="0.5"} 113
...
myapp_http_request_duration_seconds_bucket{le="+Inf"} 130
myapp_http_request_duration_seconds_count 130
myapp_http_request_duration_seconds_sum 47.31
```

6.2. Step 2: Discovery Contract — Pod Annotations

Pods opt into scraping via well-known Prometheus annotations. The collector's `relabel_configs` (next step) use them to discover scrape targets:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  template:
    metadata:
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/path:   "/metrics"
        prometheus.io/port:   "8000"
        prometheus.io/scheme: "http"
```

A pod without `prometheus.io/scrape: "true"` is invisible to the collector — annotations are the explicit opt-in.

6.3. Step 3: Collector Receiver with Kubernetes Service Discovery

The collector's `prometheus` receiver uses `kubernetes_sd_configs` to find pods and `relabel_configs` to convert annotations into scrape targets:

```
receivers:
  prometheus:
    config:
      scrape_configs:
        - job_name: "kubernetes-pods"
          scrape_interval: 15s
          kubernetes_sd_configs:
            - role: pod
              namespaces:
                names:
                  - app-prod
                  - app-stage
          relabel_configs:
            # Keep only pods explicitly annotated for scraping
            - source_labels:
[__meta_kubernetes_pod_annotation_prometheus_io_scrape]
              regex: true
              action: keep
            # Override target address with annotated port
            - source_labels: [__address__,
__meta_kubernetes_pod_annotation_prometheus_io_port]
              regex: ([^:]+)(?::\d+)?;(\d+)
              replacement: $$1:$$2
              target_label: __address__
            # Override metrics path
            - source_labels:
[__meta_kubernetes_pod_annotation_prometheus_io_path]
              regex: (.+)
              action: replace
              target_label: __metrics_path__
            # Drop pods that are Failed or Succeeded (not running)
            - source_labels: [__meta_kubernetes_pod_phase]
              regex: (Failed|Succeeded)
              action: drop
```

The collector needs RBAC to list pods cluster-wide — see **OTEL-03 — Collector Deployment** for the `ClusterRole` and `ClusterRoleBinding` shape.

6.4. Step 4: Cardinality Control via metric_relabel_configs

A pod-discovery scrape will collect every `_bucket`, `_count`, `_sum`, and platform metric on the target. Without filtering, an OTel-instrumented Python app can produce thousands of unique series per replica. `metric_relabel_configs` runs **after** scrape and **before** the pipeline, so it is the cheapest place to drop noise:

```
metric_relabel_configs:
  # Keep only metric families this collector cares about
  - source_labels: [__name__]
    regex:
(myapp_.*|webservice_.*|process_.+|go_.+|jvm_.+|http_server_requests_.+|up)
    action: keep
  # Keep only labels that downstream queries actually use
  - regex: (le|method|status_code|route|pod|namespace|cluster|service_name)
    action: labelkeep
```

The keep-list is a deliberate allow-list — easier to extend than to chase down new noisy series later. Match against `__name__` first (cheapest), then narrow labels.

6.5. Step 5: Full Pipeline

The processor pipeline shapes scraped Prometheus data for Dynatrace ingest. Order matters: `memory_limiter` first protects the collector under load; `batch` last reduces network round-trips:

```
processors:
  memory_limiter:
    check_interval: 1s
    limit_percentage: 75
    spike_limit_percentage: 15

  cumulativetodelta:
    max_staleness: 3m

  k8sattributes:
    extract:
      metadata:
        - k8s.pod.name
        - k8s.namespace.name
        - k8s.deployment.name
        - k8s.node.name
        - k8s.cluster.uid
        - k8s.container.name
    pod_association:
      - sources:
          - from: resource_attribute
            name: k8s.pod.ip

  transform:
    metric_statements:
      - context: resource
        statements:
          - set(attributes["k8s.workload.name"],
attributes["k8s.deployment.name"]) where
IsString(attributes["k8s.deployment.name"])
          - set(attributes["k8s.workload.kind"], "deployment") where
IsString(attributes["k8s.deployment.name"])

  batch/metrics:
    send_batch_max_size: 5000
    send_batch_size: 500
    timeout: 10s

exporters:
  otlphttp:
    endpoint: ${env:DT_ENDPOINT}
    headers:
      Authorization: Api-Token ${env:DT_API_TOKEN}

service:
  pipelines:
    metrics:
```

```
      receivers: [prometheus]
      processors: [memory_limiter, cumulativetodelta, k8sattributes,
transform, batch/metrics]
      exporters: [otlphttp]
```

Why each processor matters:

Processor	Purpose
`memory_limiter`	Sheds load before the collector OOMs. First in the pipeline.
`cumulativetodelta`	Prometheus emits cumulative counters; Dynatrace requires delta temporality, and its instrument mapping gives a cumulative Counter no Dynatrace metric type. Convert before export.
`k8sattributes`	Enriches each metric with pod / deployment / namespace / node so Dynatrace's entity model can join the data.
`transform`	Normalizes workload metadata — `k8s.workload.{name,kind,uid}` are derived from whichever workload kind (deployment / statefulset / daemonset / etc.) owns the pod.
`batch/metrics`	Reduces export overhead. `send_batch_size: 500` keeps batches small enough for OTLP/HTTP without hitting tenant-side payload limits.

The `otlphttp` exporter posts to `https://<tenant>.live.dynatrace.com/api/v2/otlp/v1/metrics`. The `Api-Token` auth scheme is for classic API Tokens (prefix `dt0c01.*`); Platform Tokens use `Bearer ${TOKEN}` instead. See **OTEL-07 — Dynatrace Integration** for the full token / endpoint / scope matrix.

6.6. Operator-Managed vs Raw Deployment

The collector itself can be deployed two ways:

Deployment style	Spec kind	What you write	What manages the collector
Raw	`Deployment` + `ConfigMap`	Container image + collector YAML in a ConfigMap	You — manual rollout, RBAC, lifecycle

Deployment style	Spec kind	What you write	What manages the collector
Operator-managed	`OpenTelemetryCollector` (CRD)	Collector config inline in the custom resource	The OpenTelemetry Operator reconciles Deployment, ConfigMap, ServiceAccount; cluster-scoped RBAC for pod discovery is provisioned separately (OTEL-03 §4)

In community practice, the operator-managed pattern is preferred once you run more than one collector — it owns rollout, auto-restart on config change, and OTLP-auto-instrumentation injection for application pods. The trade-off is one more controller in the cluster.

See **OTEL-03 — Collector Deployment** for both deployment shapes side-by-side.

6.7. The Reverse Direction: Exposing OTel Metrics as Prometheus

If an existing Prometheus stack must continue scraping the application, the OTel SDK can expose itself as a Prometheus endpoint instead of pushing OTLP:

```
from opentelemetry.exporter.prometheus import PrometheusMetricReader
from prometheus_client import start_http_server

# Start Prometheus endpoint
start_http_server(8080)

# Use Prometheus reader
reader = PrometheusMetricReader()
provider = MeterProvider(metric_readers=[reader])
```

This path is rarer in greenfield Dynatrace deployments — it preserves the Prometheus scrape contract at the cost of duplicating the metric pipeline.

> **Sources:**
>
> - Prometheus Receiver (OpenTelemetry Collector Contrib GitHub)
> - k8sattributes processor (OpenTelemetry Collector Contrib GitHub)
> - Cumulative to Delta processor (OpenTelemetry Collector Contrib GitHub)
> - Ingest OTLP metrics (DT docs)

> - About OTLP metrics ingest (DT docs) — *"The Dynatrace backend exclusively works with delta values and requires the respective aggregation temporality."*
> - OpenTelemetryCollector CRD reference (opentelemetry-operator GitHub) — *"When set, the operator will not automatically create a ServiceAccount for the collector."*

```
// Query OTel metrics in Dynatrace
// Dimension names follow the stable semantic conventions:
http.request.method,
// not the pre-1.0 http.method (which groups everything into one null series).
timeseries avg(http.server.request.duration), from:-1h, by:
{http.request.method}
| limit 10
```

```
// Request count by status — from spans, not from the duration metric.
//
// count(http.server.request.duration) is NOT the number of requests: it
counts data
// points (its rollup is "availability"). On 10/02/2026 it returned 112 for
both status
// codes of one service over an hour — one per series per minute, whatever the
traffic.
// Without Advanced OTLP metric dimensions, an explicit-bucket histogram is
stored as a
// Dynatrace Counter, so the per-request count is not in the metric. Count the
server spans.
// gRPC services have no http.response.status_code, so they appear with a null
status.
fetch spans, from:-1h
| filter span.kind == "server" and dt.openpipeline.source ==
"/api/v2/otlp/v1/traces"
| summarize {requests = count()}, by:{dt.service.name,
http.response.status_code}
| sort requests desc
| limit 10
```

7. Best Practices

Naming Conventions

Pattern	Example	Description
`<domain>.<component>.<metric>`	`myapp.orders.processed`	Hierarchical naming; reuse semantic-convention names only with their defined instrument and unit
Use lowercase	`request_count` not `RequestCount`	Consistency
Include units	`duration_ms`, `size_bytes`	Clarity

Performance Tips

Tip	Reason
Limit attribute cardinality	Memory, query performance
Use appropriate export interval	Balance freshness vs. load
Batch metrics	Reduce network calls
Use observable for system stats	Avoid polling in hot path

What to Measure

Category	Metrics
RED	Rate, Errors, Duration (for services)
USE	Utilization, Saturation, Errors (for resources)
Business	Revenue, conversions, user actions

Summary

In this notebook, you learned:

- Metric types: Counter, UpDownCounter, Histogram, Gauge
- Sync vs async instruments
- Creating and recording metrics
- Attribute best practices and cardinality
- Views and aggregation configuration
- Prometheus integration patterns

References

- OTel Metrics Specification
- OTel Python Metrics
- Dynatrace OTLP Metrics Ingest
- Configure OTLP Metrics
- About OTLP metrics ingest (DT docs) — instrument-to-metric-type mapping
- HTTP metrics semantic conventions (opentelemetry.io)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.